



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Cheatsheet Completo: TypeScript (Fundamentos, Interfaces, Genéricos y Utilidades Avanzadas)

Resumen

Esta guía rápida y cheatsheet proporciona una referencia completa del superconjunto tipado **TypeScript**, abarcando el sistema de tipos estático, interfaces, alias de tipo, genéricos (`<T>`), estrechamiento de tipos (*Type Narrowing*), tipos de utilidad avanzadas y configuración del compilador `tsc`.

1. Tipos Primarios y Especiales

```

// Tipos Primarios
const nombre: string = 'Juan';
const edad: number = 22;
const esEstudiante: boolean = true;
const valores: number[] = [1, 2, 3];
const tupla: [string, number] = ['UCR', 2026];

// Enum (Enumeracion explicita)
enum EstadoReserva {
    PENDIENTE = 'PENDIENTE',
    CONFIRMADA = 'CONFIRMADA',
    CANCELADA = 'CANCELADA'
}
const estadoActual: EstadoReserva = EstadoReserva.CONFIRMADA;

// Tipos Especiales
let desconocido: unknown = 'Hola'; // Seguro (requiere verificacion antes de usar)
let cualquiera: any = 123; // EVITAR (Deshabilita el chequeo de tipos de TS)

function registrarLog(msg: string): void {
    console.log(msg);
}

function lanzarErrorFatal(msg: string): never {
    throw new Error(msg); // Nunca retorna un valor
}

```

2. Interfaces vs Type Aliases

```

// Interface (Ideal para contratos de Objetos y Clases - Extensible)
interface Usuario {
    readonly id: number; // Propiedad de solo lectura
    nombre: string;
    email: string;
    edad?: number; // Propiedad opcional
}

// Extensión de Interfaces
interface Admin extends Usuario {
    nivelAcceso: number;
}

```

```
// Type Alias (Ideal para Uniones, Intersecciones y Primitivos)
type Rol = 'ADMIN' | 'OPERADOR' | 'CLIENTE';
type Coordenada = [number, number];

// Interseccion (&) y Union (|)
type UsuarioConRol = Usuario & { rol: Rol };
```

3. Genéricos / Tipos Genéricos (<T>)

Permiten escribir componentes y funciones reutilizables que funcionan sobre múltiples tipos manteniendo la seguridad de tipos:

```
// Funcion Generica
function obtenerPrimerElemento<T>(array: T[]): T | undefined {
  return array.length > 0 ? array[0] : undefined;
}

const primerNumero = obtenerPrimerElemento<number>([10, 20, 30]); // number
const primerTexto = obtenerPrimerElemento<string>(['a', 'b']); // string

// Interface Generica (Respuesta API Homogenea)
interface RespuestaAPI<T> {
  status: number;
  mensaje: string;
  data: T;
}

// Uso con un DTO específico
interface ClienteDTO {
  id: number;
  empresa: string;
}

type RespuestaCliente = RespuestaAPI<ClienteDTO>;

// Restriccion en Genericos (extends)
function obtenerPropiedadId<T extends { id: number }>(objeto: T): number {
  return objeto.id;
}
```

4. Estrechamiento de Tipos (*Type Narrowing & Type Guards*)

```
// 1. Uso de typeof e instanceof
function procesarValor(val: string | number | Date) {
  if (typeof val === 'string') {
    console.log(val.toUpperCase());
  } else if (val instanceof Date) {
    console.log(val.toISOString());
  } else {
    console.log(val.toFixed(2));
  }
}

// 2. Custom Type Guard (Predicado de Tipo: pet is Dog)
interface Perro { guau: () => void; }
interface Gato { miau: () => void; }

function esPerro(mascota: Perro | Gato): mascota is Perro {
  return (mascota as Perro).guau !== undefined;
}

function hacerSonido(mascota: Perro | Gato) {
  if (esPerro(mascota)) {
    mascota.guau();
  } else {
    mascota.miau();
  }
}
```

5. Tipos de Utilidad Incorporados (*Utility Types*)

TypeScript provee transformaciones de tipo integradas muy útiles:

```
interface Tarea {
  id: number;
  titulo: string;
  descripcion: string;
  completada: boolean;
}
```

```
// 1. Partial<T>: Hace que todas las propiedades sean opcionales
type TareaActualizar = Partial<Tarea>;

// 2. Required<T>: Hace que todas las propiedades sean obligatorias
type TareaCompleta = Required<Tarea>;

// 3. Readonly<T>: Hace que todas las propiedades sean de solo lectura
type TareaInmutable = Readonly<Tarea>;

// 4. Pick<T, K>: Selecciona solo un subconjunto de claves K
type TareaResumida = Pick<Tarea, 'id' | 'titulo'>;

// 5. Omit<T, K>: Elimina un subconjunto de claves K
type TareaSinId = Omit<Tarea, 'id'>;

// 6. Record<K, T>: Construye un mapa de claves K con valores T
type MapaRoles = Record<string, boolean>;
```

6. Configuración Recomendada de `tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "noImplicitAny": true,
    "strictNullChecks": true,
    "noImplicitReturns": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "outDir": "./dist"
  },
  "include": ["src/**/*"]
}
```

7. Consejos Prácticos

■ **Importante (Modo Estricto):** Active la bandera `"strict": true` en `tsconfig.json`. Esto habilita `strictNullChecks` y deshabilita los `any` implícitos, evitando errores `Cannot read properties of undefined` en tiempo de ejecución.

■ **Consejo Práctico (Uso de `unknown` vs `any`):** Prefiera `unknown` sobre `any`. `unknown` le obliga a realizar un chequeo o estrechamiento de tipo (*Type Narrowing*) antes de realizar operaciones sobre el valor, garantizando la seguridad de tipos.